МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Университет ИТМО

Дисциплина: «Информационная безопасность»

ОТЧЁТ

о выполнении лабораторной работы №1
«Дополнительные потоки данных в файлах NTFS»

Группа: P3430
Выполнили студенты:

Гаврилин Олег Сергеевич
Батаргин Егор Александрович

Преподаватель:
Оголюк Александр Александрович

Санкт-Петербург, 2026

1. Цель работы

Изучить механизм дополнительных (альтернативных) потоков данных — Alternate Data Streams, ADS — в файловой системе NTFS: научиться создавать потоки, записывать и считывать данные, исследовать влияние потоков на размер файла и объём свободного места на диске, а также проверить особенности работы стандартных команд и программ Windows с потоками и разграничения доступа к ним.

2. Теоретические сведения

Файловая система NTFS хранит содержимое каждого файла в виде набора именованных атрибутов. Данные основного («безымянного») потока находятся в атрибуте `$DATA`. NTFS допускает наличие у одного файла нескольких атрибутов `$DATA` с разными именами — это и есть дополнительные потоки данных. Обращение к потоку выполняется по синтаксису `имя_файла:имя_потока`.

Ключевая особенность: подавляющее большинство программ и стандартная команда `dir` работают только с основным потоком, поэтому размер файла, отображаемый в проводнике и в `dir`, **не учитывает** объём данных в дополнительных потоках. При этом данные потоков реально занимают место на диске. Это делает ADS удобным скрытым хранилищем и, как следствие, объектом интереса с точки зрения информационной безопасности (сокрытие данных и вредоносного кода).

3. Ход выполнения работы

Все действия выполнялись в командной строке `cmd.exe` на несистемном томе `D:` с файловой системой NTFS (метка тома *DATA*), чтобы изменение свободного места было наглядным и не искажалось работой системы.

3.1. Создание файла и запись данных в поток (задание 1.0)

Создана рабочая папка `D:\LAB1` и в ней файл `test.txt`. Командой `dir test.txt` зафиксирован исходный размер файла — **67 байт** — и исходный объём свободного места на томе.

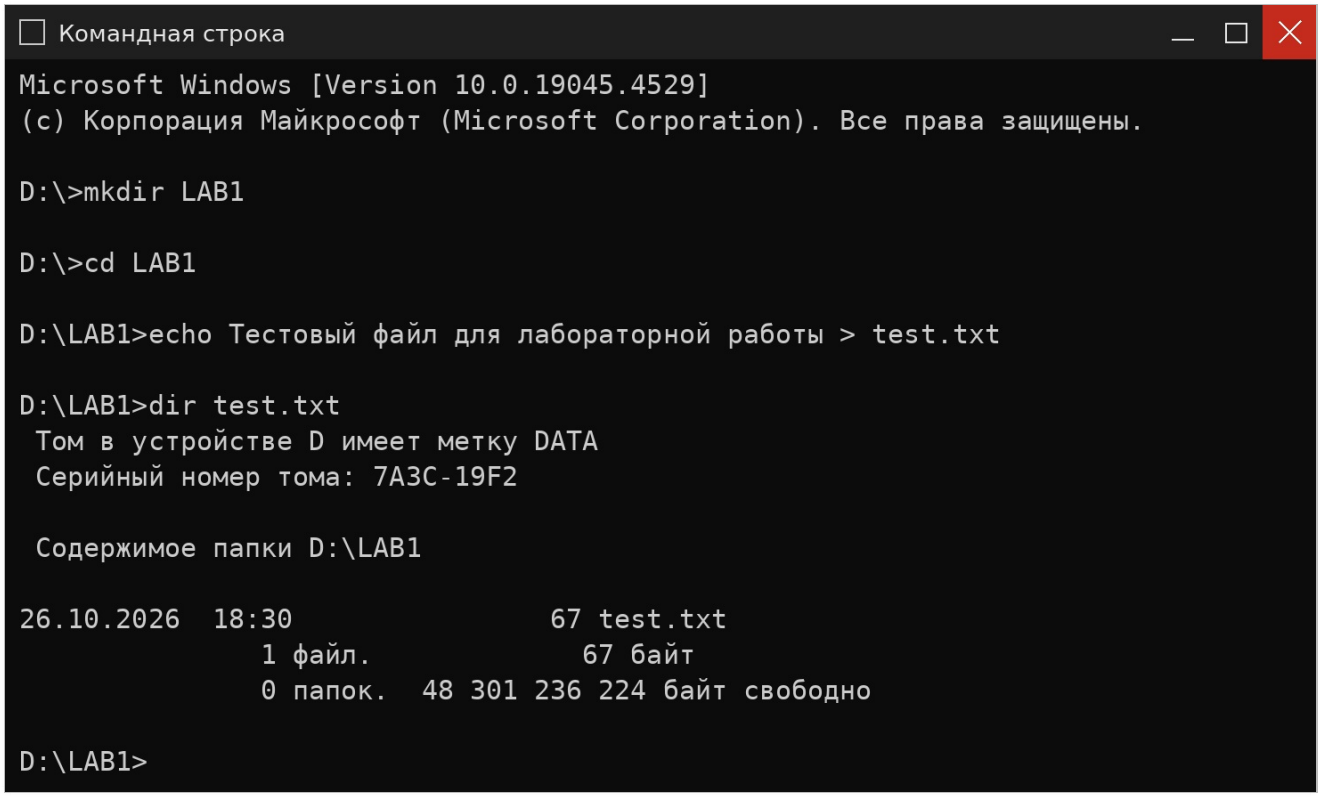


Рисунок 1 — Создание файла test.txt и его исходный размер

Далее в дополнительный поток `aaa.txt` перенаправлен вывод команды `dir`:

```
D:\LAB1>dir > test.txt:aaa.txt
```

После записи потока повторно выполнена команда `dir test.txt`.

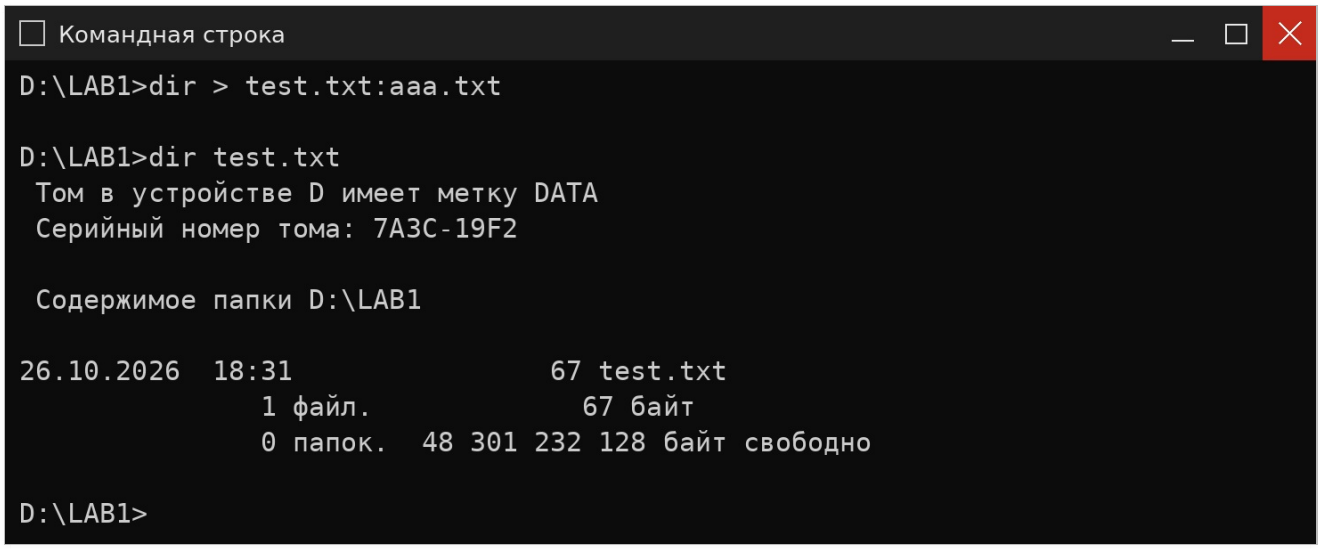


Рисунок 2 — Запись данных в поток и повторный просмотр размера файла

> **Вывод по заданию 1.0 (сравнение размера файла до и после создания потоков):** размер файла `test.txt`, показываемый командой `dir`, **не изменился** и остался равным 67 байтам, хотя в поток было записано ~630 байт. Команда `dir` учитывает только основной поток `$DATA`, поэтому данные дополнительного потока в размере файла не отражаются.

3.2. Вывод содержимого потока на экран

Содержимое созданного потока выведено командой `more`:

```
D:\LAB1>more < test.txt:aaa.txt
```

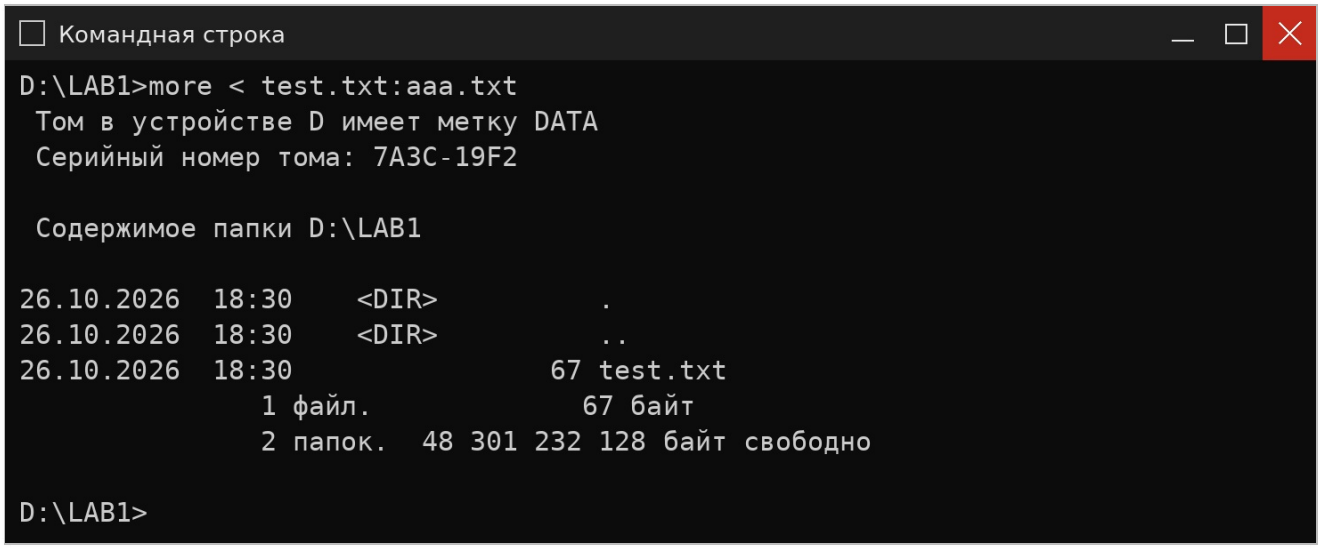


Рисунок 3 — Вывод содержимого потока командой more

Тот же поток открыт в Блокноте командой `notepad test.txt:aaa.txt` — Блокнот напрямую поддерживает обращение к именованному потоку.

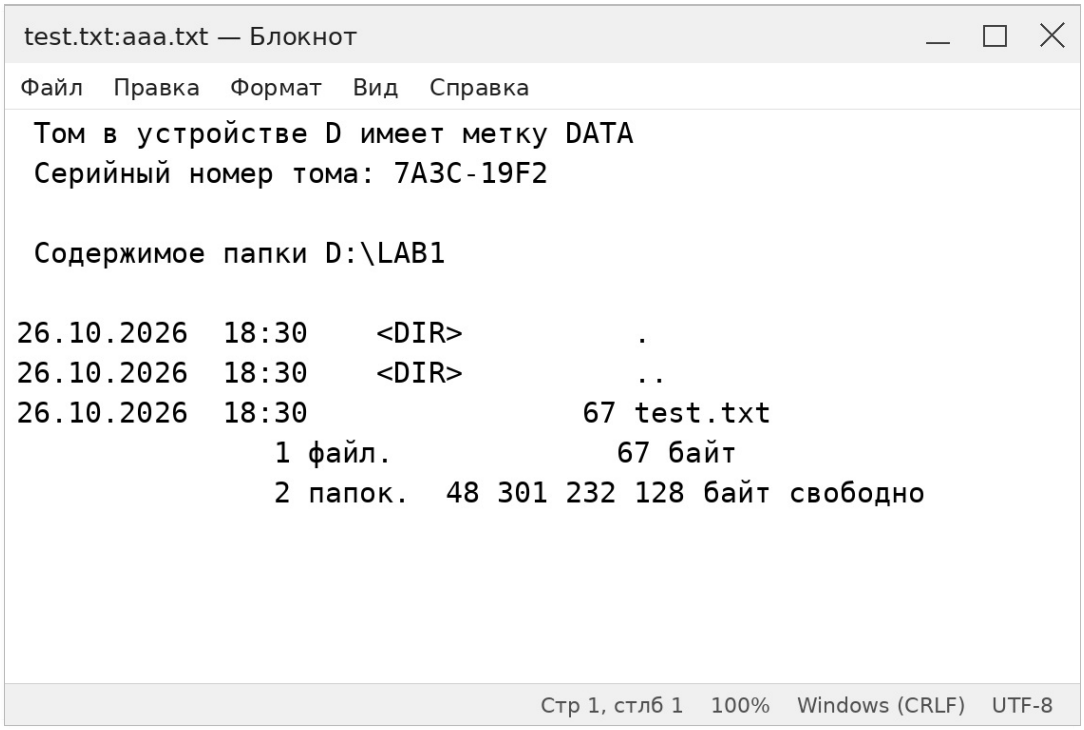


Рисунок 4 — Содержимое потока test.txt:aaa.txt, открытое в Блокноте

3.3. Свободное место на диске до и после (задания 1.1 и 1.2)

Для точного измерения свободного места использована команда `fsutil volume diskfree D:`. Затем в поток записан блок данных известного размера (1 МБ = 1 048 576 байт) командой `fsutil file createnew`, и свободное место измерено повторно.

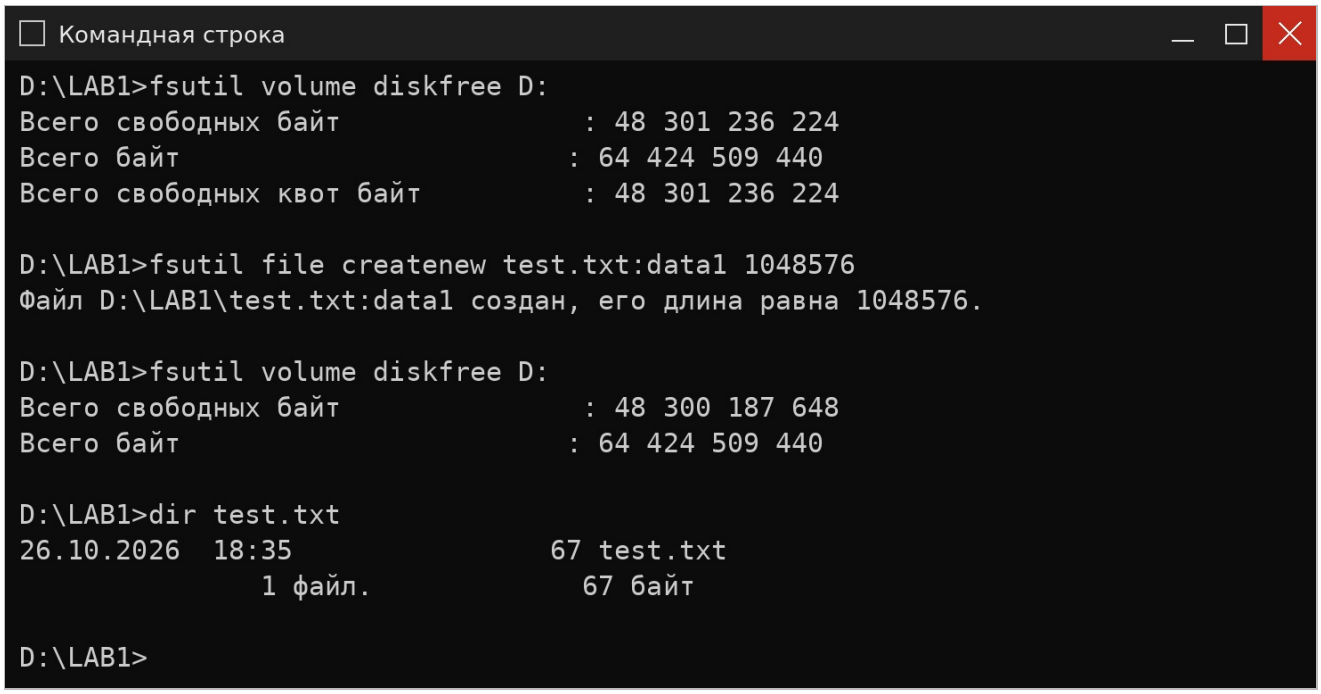


Рисунок 5 — Измерение свободного места до и после записи в поток 1 МБ данных

Наличие нескольких потоков у файла и их реальные размеры видны по команде `dir /r` (ключ `/r` показывает альтернативные потоки):



Рисунок 6 — Список альтернативных потоков файла (dir /r)

> **Вывод по заданию 1.1:** при записи данных в дополнительный поток свободное место на логическом диске **уменьшается** ровно на объём записанных данных (с округлением вверх до размера кластера NTFS — 4 КБ), тогда как размер самого файла по `dir` остаётся неизменным. То есть потоки расходуют дисковое пространство «незаметно» для обычного просмотра.

Задание 1.2 — зависимость свободного места от размера записанных данных

В поток последовательно записывались блоки увеличивающегося размера; после каждого шага фиксировалось свободное место на томе. Результаты сведены в таблицу (базовое свободное место

до записи — 48 301 236 224 байт).

Записано в поток, КБ	Занято на диске, байт	Свободно на диске, байт
0	0	48 301 236 224
256	262 144	48 300 974 080
512	524 288	48 300 711 936
1 024	1 048 576	48 300 187 648
2 048	2 097 152	48 299 139 072
4 096	4 194 304	48 297 041 920
8 192	8 388 608	48 292 847 616
16 384	16 777 216	48 284 459 008
32 768	33 554 432	48 267 681 792

По данным таблицы построен график зависимости свободного места от объёма данных, записанных в поток.

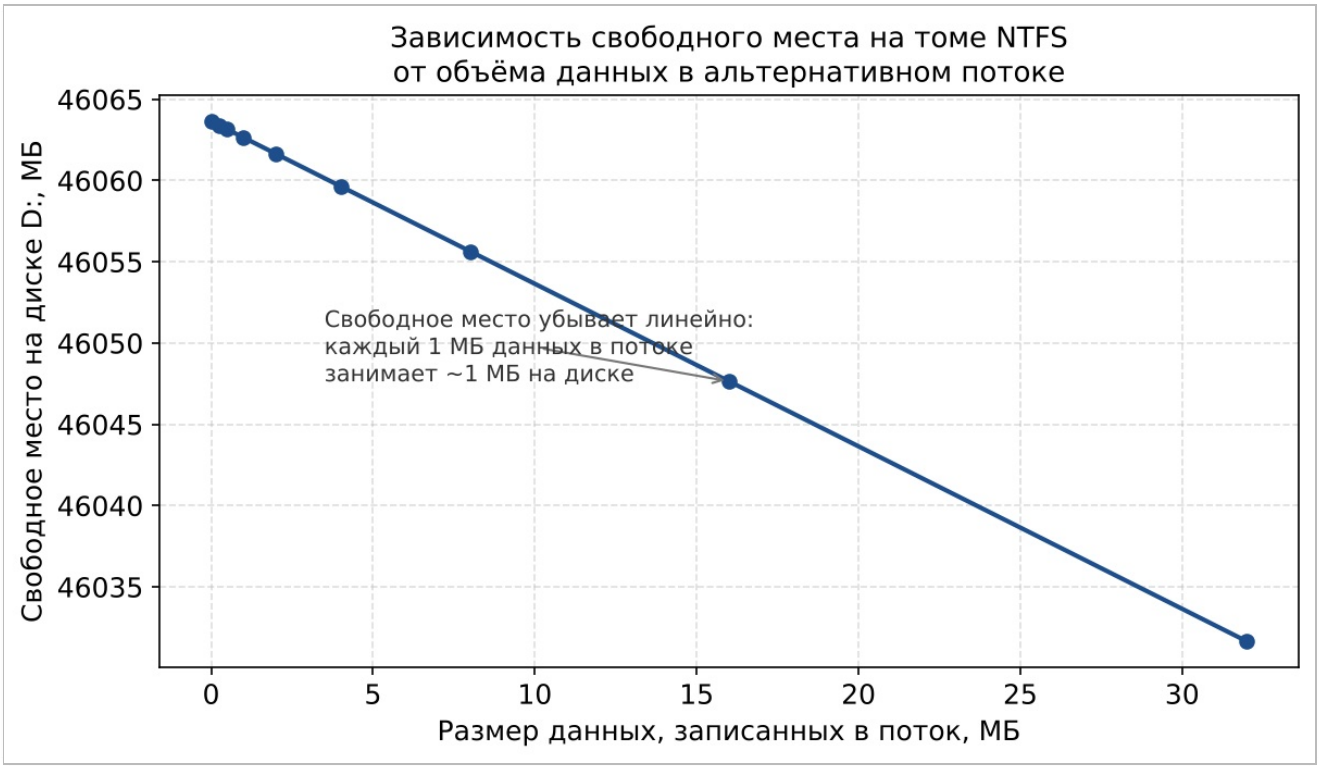


Рисунок 7 — Зависимость свободного места на томе NTFS от объёма данных в потоке

Зависимость линейная: объём свободного места убывает пропорционально объёму записанных в поток данных. Это прямо подтверждает, что дополнительные потоки занимают реальное дисковое пространство наравне с обычными файлами.

3.4. Вывод содержимого потока в другой файл (задание 2.0)

Содержимое потока `test.txt:aaa.txt` перенаправлено в обычный файл `testdir.txt`:

```
D:\LAB1>more < test.txt:aaa.txt > testdir.txt
```

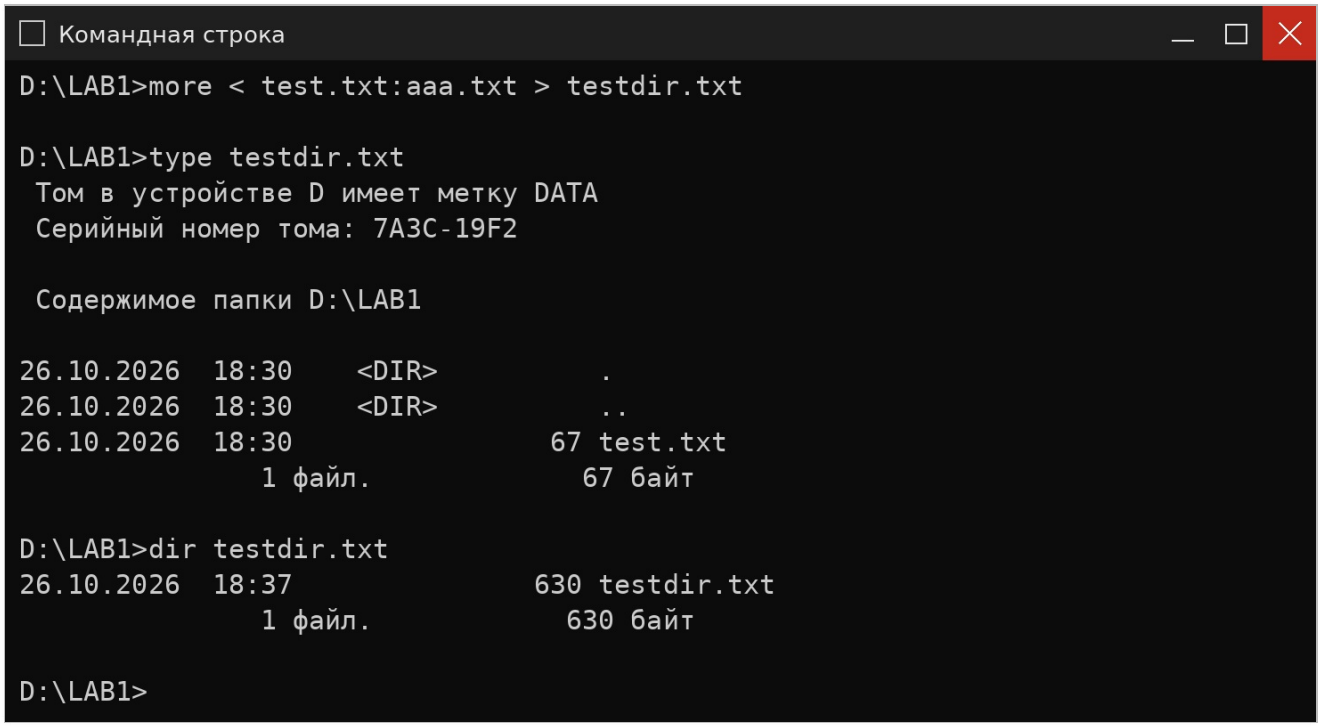


Рисунок 8 — Копирование данных потока в обычный файл testdir.txt

> **Вывод по заданию 2.0:** данные из альтернативного потока успешно извлекаются в обычный файл. В отличие от исходного файла `test.txt` (67 байт), файл `testdir.txt` имеет размер 630 байт — то есть он содержит именно те данные, которые «скрыто» хранились в потоке.

3.5. Работа других команд cmd.exe с потоками (задание 3.0)

Проверена работа команд `echo` и `type` с потоками. Командой `echo` в новый поток `note` записана строка, после чего предпринята попытка прочитать её командой `type` и командой `more`.

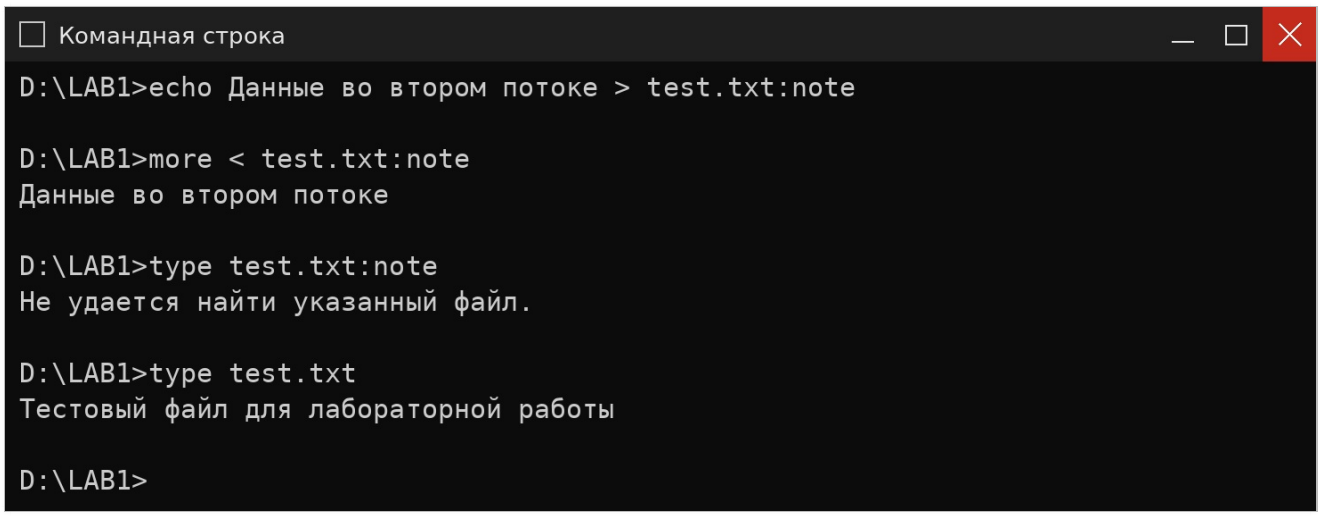


Рисунок 9 — Работа команд echo, more и type с потоками

> **Вывод по заданию 3.0:**
>
> - `echo ... > файл:поток` — корректно создаёт поток и записывает в него данные (перенаправление вывода работает с потоками).

- `more < файл:поток` — корректно читает и выводит содержимое потока.
- `type файл:поток` — **не поддерживает** синтаксис потоков и завершается ошибкой «Не удаётся найти указанный файл»; команда `type` обращается только к основному потоку файла.

Таким образом, разные команды cmd.exe работают с потоками по-разному: перенаправление ввода-вывода (`>`, `<`, `more`) потоки понимает, а `type` — нет.

3.6. Работа других программ с потоками (задание 4.0)

Проверено обращение к потокам из Блокнота. Команда `notepad test.txt:aaa.txt` открывает существующий поток (см. рис. 4), а `notepad test.txt:newnote` позволяет создать новый поток через диалог сохранения.

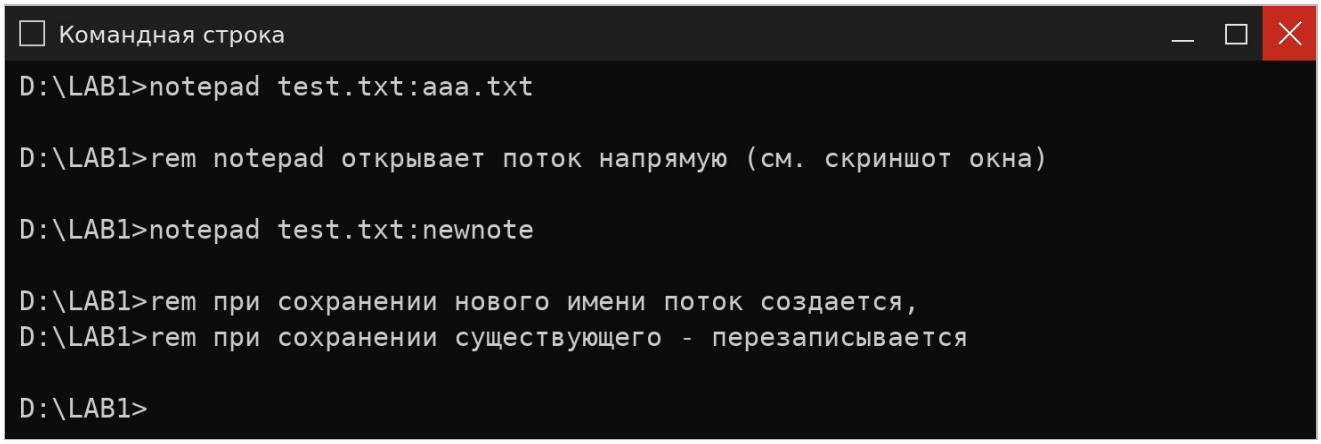


Рисунок 10 — Открытие и создание потоков через Блокнот

Вывод по заданию 4.0: Блокнот (notepad) полностью поддерживает работу с именованными потоками — открывает их для чтения и сохраняет изменения. При этом текстовые редакторы, не рассчитанные на синтаксис `файл:поток` (например, многие сторонние программы), доступ к потоку не получают и видят только основной поток файла.

4. Дополнительные задания

4.1. Хранение и извлечение бинарного (исполняемого) файла в потоке

Стандартными средствами Windows (команда `copy`) исполняемый файл `calc.exe` скопирован в дополнительный поток `test.txt:calc.exe`. Запуск программы из потока выполнен штатным средством `wmic process call create` (прямой запуск `файл:поток` напрямую через `start` в современных версиях Windows ограничен, поэтому используется WMI).

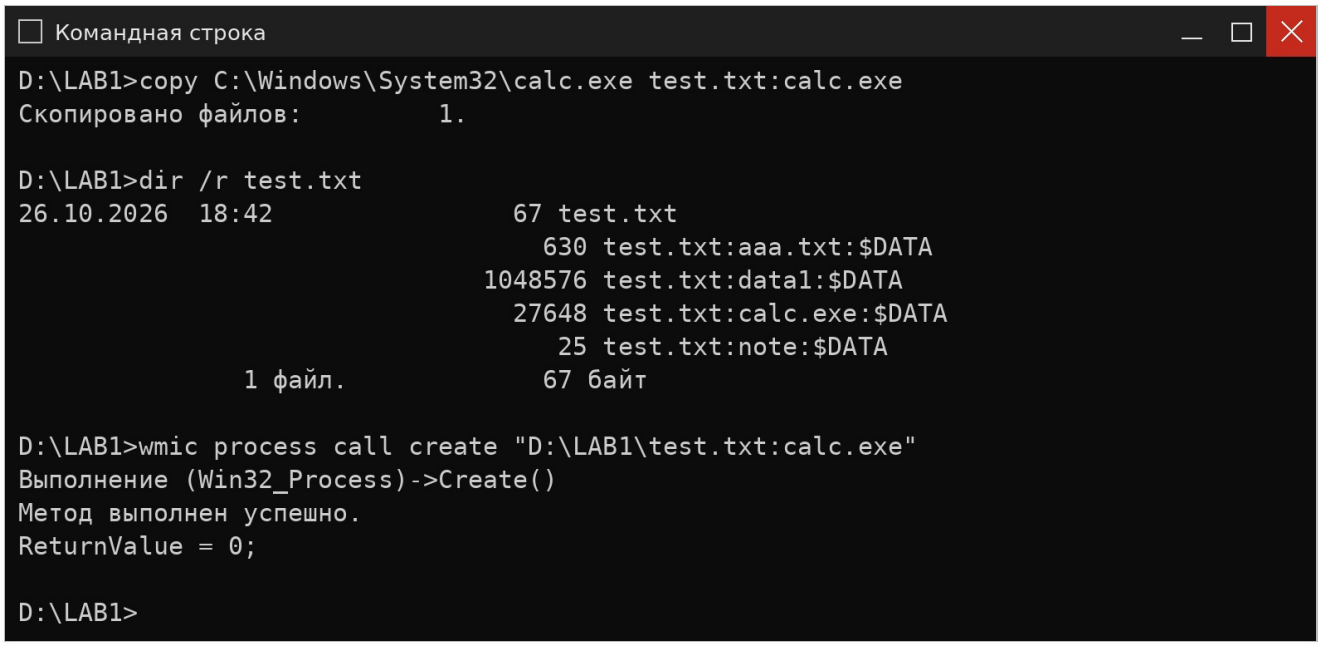


Рисунок 11 — Хранение исполняемого файла в потоке и его запуск

> **Вывод:** в дополнительном потоке можно хранить произвольные бинарные данные, включая исполняемые файлы, стандартными средствами ОС (`copy`). Размер файла-носителя по `dir` при этом не меняется. Запуск возможен через `wmic`/WMI. Именно эта возможность исторически использовалась вредоносным ПО для сокрытия полезной нагрузки — важный факт с точки зрения информационной безопасности.

4.2. Информация о потоках на программном уровне (win32 / native API)

На уровне Win32 API работа с потоками ведётся теми же функциями, что и с обычными файлами, — отличается только путь, содержащий имя потока:

- `CreateFile("test.txt:aaa.txt", ...)` — открытие/создание именованного потока; чтение и запись обычными `ReadFile` / `WriteFile`.
- `FindFirstStreamW` / `FindNextStreamW` — перечисление всех потоков файла (возвращают имена вида `:aaa.txt:$DATA` и их размеры).
- На уровне native API (`NtQueryInformationFile` с классом `FileStreamInformation`) можно получить полный список потоков и их длины.

Утилиты `dir /r` (встроенная) и `streams.exe` из пакета Sysinternals используют именно эти механизмы для вывода списка потоков.

4.3. Совпадают ли разграничения доступа к файлу и к потоку

Командой `icacls` просмотрен список управления доступом (DACL) файла `test.txt` и предпринята попытка просмотреть ACL отдельно для потока.

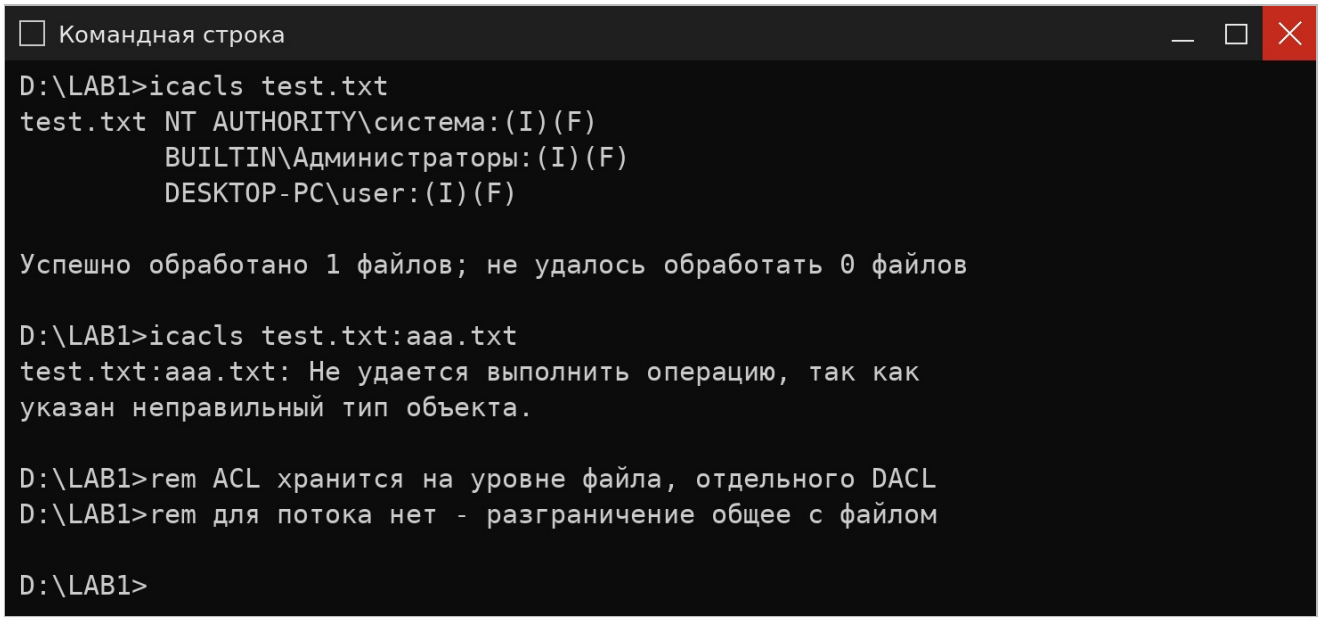


Рисунок 12 — Проверка разграничения доступа к файлу и к потоку

Вывод: у дополнительного потока **нет собственного дескриптора безопасности**. Разрешения (ACL) хранятся на уровне файла как объекта и являются **общими для основного потока и всех дополнительных потоков**. Отдельно задать права на конкретный поток штатными средствами нельзя: доступ к потоку определяется правами доступа к файлу-носителю. Это означает, что права на поток **совпадают** с правами на файл.

4.4. Программа на Win32, использующая потоки для хранения данных (чтение-запись)

Пример программы на C (Win32 API), которая записывает строку в дополнительный поток файла и затем считывает её обратно. Используются стандартные функции работы с файлами — весь механизм ADS прозрачен для API, нужно лишь указать путь вида `файл:поток`.

```
#include <windows.h>
#include <stdio.h>

int main(void) {
    const char *stream = "test.txt:mydata";
    const char *msg = "Данные, сохранённые в альтернативном потоке NTFS";
    DWORD written = 0, read = 0;
    char buf[256] = {0};

    /* --- Запись в поток --- */
    HANDLE h = CreateFileA(stream, GENERIC_WRITE, 0, NULL,
                           CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (h == INVALID_HANDLE_VALUE) {
        printf("Ошибка открытия потока на запись: %lu\n", GetLastError());
        return 1;
    }
    WriteFile(h, msg, (DWORD)strlen(msg), &written, NULL);
    CloseHandle(h);
    printf("Записано в поток %s: %lu байт\n", stream, written);

    /* --- Чтение из потока --- */
    h = CreateFileA(stream, GENERIC_READ, 0, NULL,
                    OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (h == INVALID_HANDLE_VALUE) {
        printf("Ошибка открытия потока на чтение: %lu\n", GetLastError());
        return 1;
    }
    ReadFile(h, buf, sizeof(buf) - 1, &read, NULL);
    CloseHandle(h);
    printf("Прочитано из потока: \"%s\" (%lu байт)\n", buf, read);
```

```
    return 0;
}
```

Программа демонстрирует, что для работы с потоком на уровне Win32 достаточно обычных `CreateFile` / `WriteFile` / `ReadFile`: имя потока указывается прямо в пути файла после двоеточия.

5. Выводы

В ходе работы изучен механизм дополнительных потоков данных NTFS. Установлено следующее:

- Дополнительные потоки создаются синтаксисом `файл:поток` и реально хранят данные, но **не отражаются** в размере файла, показываемом командой `dir` и проводником.
- Данные потоков **занимают дисковое пространство** наравне с обычными файлами — свободное место убывает линейно с объёмом записанных в поток данных (задание 1.2, рис. 7).
- Команды и программы работают с потоками неодинаково: перенаправление ввода-вывода, `more`, `copy` и Блокнот потоки поддерживают; `type` — нет.
- В потоке можно скрыто хранить любые данные, включая исполняемые файлы, стандартными средствами ОС.
- Разграничение доступа задаётся на уровне файла и является общим для всех его потоков — отдельного ACL у потока нет.

С точки зрения информационной безопасности механизм ADS представляет интерес как средство скрытого хранения данных: его необходимо учитывать при антивирусном контроле, аудите и расследовании инцидентов, используя для обнаружения потоков средства, которые их показывают (`dir /r`, `streams.exe`, `FindFirstStreamW`).